Platform core

Calibrated DSP, one feature contract, two implementations that agree

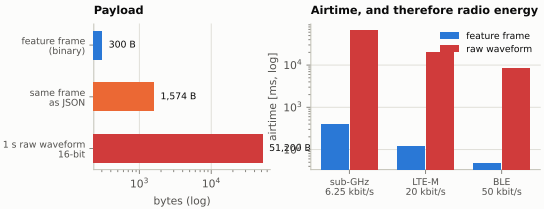
Python core	64 tests, 28 modules
C twin	1 049 golden-vector checks
Code size	9.6 kB at `-Os`
RAM	85 kB for a 4096-point pipeline
Wire frame	300 B, CRC-32, versioned
End-to-end POC	`app.py --replay`

Branch `platform/core`.

Every number in this deck is produced by running the code at build time.

Every number is also **from simulated data**. No model here has been validated on measured data; no firmware here has run on hardware. Both gaps are stated where they matter and listed in `docs/07_production_readiness.md`.

The decision everything else follows from



Features on the radio, waveforms only on demand.

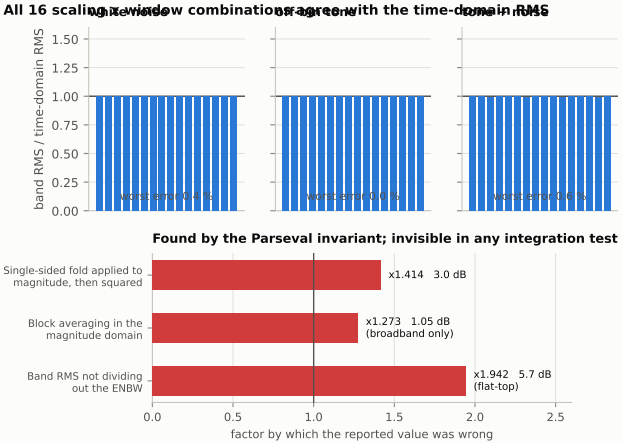
A 1 s, 25.6 kHz, 16-bit acquisition is 51 kB. The feature frame is 300 B — **170× smaller**. Radio energy dominates a battery sensor's budget, so that ratio is the battery life.

Consequences, all of them forced:

- The frame must support *diagnosis*, not just detection — hence 63 features including bearing-frequency evidence, not 6 summary statistics. The information is destroyed at the device; the cloud cannot reconstruct it.
- The edge needs a real FFT, envelope demodulation and a model. Hence `cdsp/`.
- Edge and cloud must compute *identical* features, or trends break at the boundary between a live frame and a recomputed snapshot.

So what: A 170× payload ratio is not an optimisation, it is the architecture. Everything hard about this project follows from it.

Calibration is not a detail, and it is testable



The invariant

Band RMS over the full band must equal the time-domain RMS — for *every* scaling (`peak`, `rms`, `power`, `psd`) and *every* window. 16 combinations, three signal types.

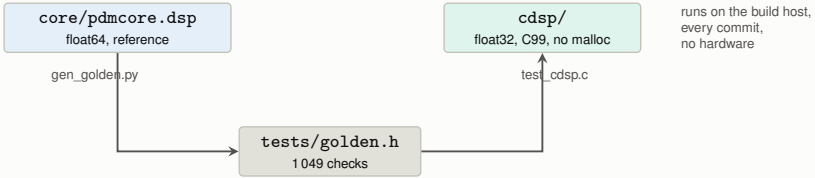
It must also read a single tone's amplitude correctly. One function, both properties: that is what forces the maths to be right rather than merely self-consistent.

Three bugs it caught

Each was invisible in an integration test, consistent across the whole codebase (so every internal comparison agreed), and would have been wrong by a constant factor **forever** — discovered the first time somebody compared the dashboard to a calibrated handheld meter.

The $\pi/4$ one is the nastiest: averaging magnitudes instead of power biases *broadband content only*. Tones were correct. Only the noise floor was 1 dB low.

Two implementations, held together by generated vectors



Why bother. A maintenance decision must not depend on *where* the number was computed. If the MCU and the cloud disagree, every threshold, baseline and trend is quietly wrong in a way that is almost impossible to debug from a dashboard.

Tolerances are per-check and justified: the expected float32 accumulation error for that specific operation, not “close enough”. A tolerance that has to be loosened is a signal.

It caught a bug nothing else would have. The FFT twiddle table was allocated `3n/4` floats where the algorithm needs `n+2`. The split-stage table overflowed into itself and corrupted **exactly bins** $n/8 + 1 \ldots 3n/8$ of every transform.

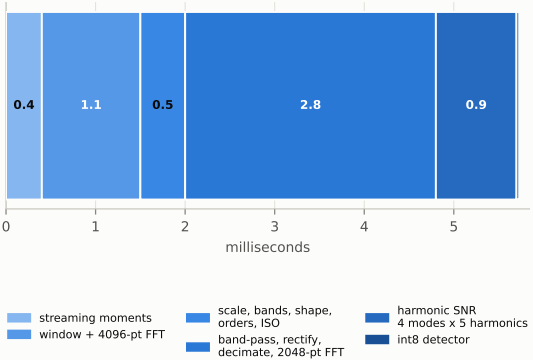
Bins outside that range were perfect. A tone sweep would have passed. Parseval over the full band would have passed. A spectrum plot would have looked plausible.

395 of 1 049 checks failed and pointed straight at it.

So what: Hence `PDM_RFFT_TWIDDLE_LEN(n)` — size the table with the macro, never by hand.

What the device actually pays

Full feature vector in ~5.7 ms (estimate: cycle counts, not measured)



Code (`-Os`)	
`pdm_envelope.o`	2 132 B
`pdm_spectrum.o`	1 996 B
`pdm_features.o`	1 556 B
`pdm_fft.o`	1 348 B
`pdm_bearing.o`	1 108 B
`pdm_model.o`	868 B
`pdm_window.o`	636 B
total	**9 644 B**

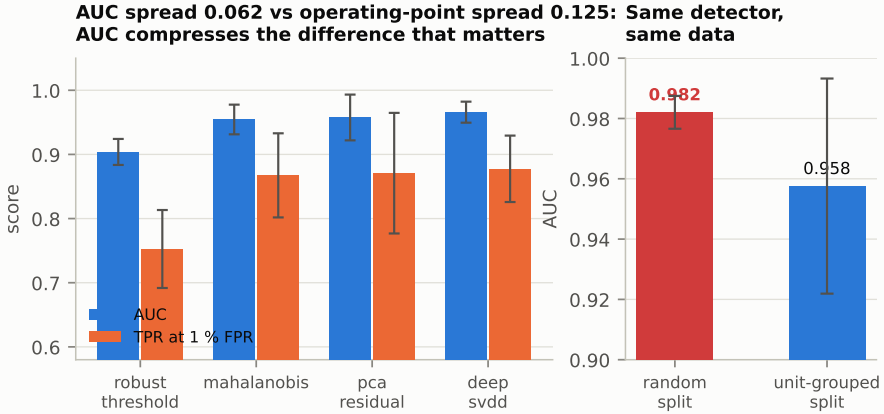
RAM for a 4096-point pipeline: **85 kB** (window, twiddle, bit-reversal, in-place FFT buffer, magnitude, spectrum, envelope, model scratch).

Fits STM32U585, nRF9160, ESP32-S3, RA8D1, Alif E7. Does *not* fit a 64 kB part; the documented fallback is `n_fft` 2048 — 43 kB, 6.25 Hz resolution instead of 3.125 Hz.

At one acquisition per hour, 5.7 ms is a **1.6 ppm** compute duty cycle. The radio and the sensor's settling time dominate, not the DSP — which is what justifies doing full envelope analysis on the device.

Code size and RAM are measured. The millisecond figures are estimates from cycle counts and CMSIS-DSP benchmarks — no board has been flashed.

Detectors, and the three things the table really says



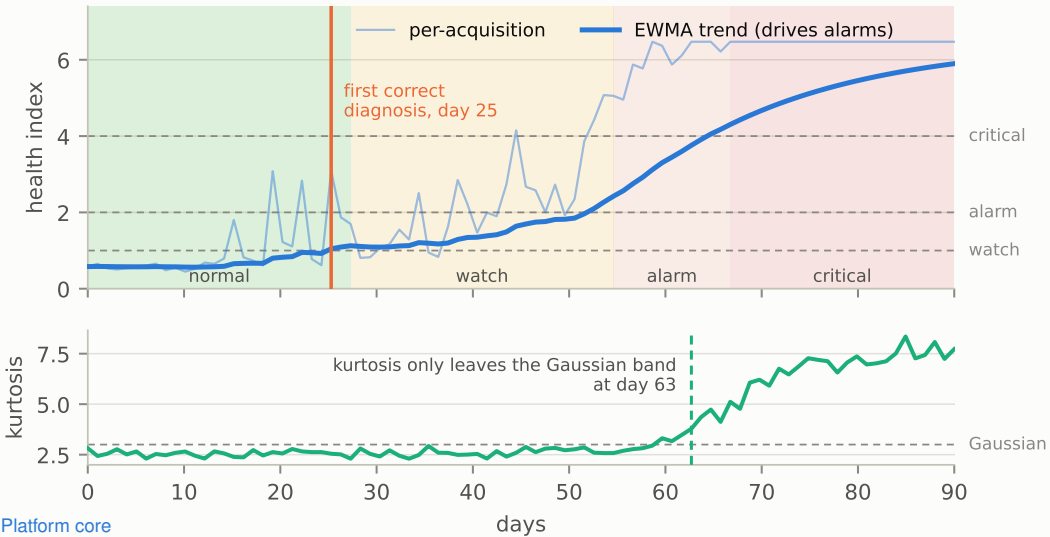
1. The top three are not separable. Within one standard deviation on every metric. Choose on explainability and footprint, not on these numbers.

2. AUC hides what matters. AUC spread 0.062; spread at 1 % FPR **0.125**. `robust_threshold` gives up 6 points of AUC and **12 of recall**.

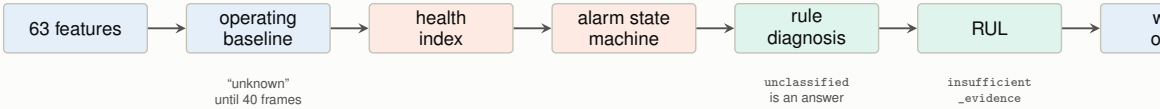
3. The split beats the model. And a leaky split looks *more* reproducible (±0.005 vs ±0.036) — which is how it survives review.

Simulated fleet: 3 machine classes × 3 units, 594 acquisitions, 7 split seeds, `leakage_report` clean on every one. ±3 % speed jitter per unit so no model can memorise a shaft frequency.

90 days to failure



The chain, and where it is allowed to abstain



Abstention is a feature.
An asset inside its commissioning window is *unknown*, not healthy, and no alarm can fire. The RUL filter returns `insufficient_evidence` far more often than a number. Diagnosis can return `elevated_vibration_unclassified`.

"The level is high and we do not yet know why" is a legitimate and useful output. A confidently wrong fault label is not.

The baseline adapts only while NORMAL.
Mounts relax, grease redistributes, ambient temperature moves 25 K with the seasons — a frozen baseline becomes a false-alarm generator.

But adaptation gated on *time* absorbs a developing fault into "normal" and hides exactly what it was built to find. Gating on *state* is what makes it safe.

Feed RUL an amplitude, not the index.
The health index is capped so the alarm logic saturates sensibly. An exponential fit to a saturating signal concludes the crossing already happened.

So prognostics consume `band.10_rms` normalised to its healthy median, and "failure" is defined as 20× the healthy high-frequency level — a statement about the asset, not about the index.

End to end, with no infrastructure

```
$ python3 cloud/ingest/app.py --replay
      day   sev    HI  trend     state  diagnosis                      RUL
     28.3  0.01  0.78   1.06     watch  -                              insufficient_evidence
     42.5  0.02  1.72   1.31     watch  bearing_outer_race (0.80)      insufficient_evidence
     56.6  0.05  5.38   2.53     alarm  bearing_outer_race (1.00)      insufficient_evidence
     70.8  0.19  6.47   4.61  critical  bearing_outer_race (1.00)      already_failed

validation checks
  corrupted CRC      -> rejected
  CLIPPED flag       -> rejected
  replayed sequence  -> deduplicated
  silent for 115 d   -> stale alarm raised
```

The device side encodes real 300-byte wire frames; the ingest side runs the same `handle_raw()` the MQTT and HTTP transports call. No broker, no database, no Docker.

The four lines at the bottom are the ones that matter. A POC that only demonstrates the happy path has not demonstrated the thing that matters: a clipped acquisition scored as an anomaly, or a store-and-forward replay re-driving the alarm machine, are the failures that destroy trust in a deployment — and they are invisible until they happen in the field.

What is not true yet

Easy to believe, and false

- **“The firmware works.”** Structurally complete, DSP verified against a reference. *Never flashed to a board.*
- **“The models are good.”** 0.90–0.97 AUC on *simulated* data. The simulator was written by the same process that wrote the features, so some agreement is self-fulfilling.
- **“The timings are measured.”** Code size and RAM are. The millisecond figures are cycle-count estimates.
- **“The power budgets are measured.”** Datasheet arithmetic. Reality is typically 20–50 % worse.
- **“The compose stack runs.”** Never started — no Docker in this environment.

Full assessment with a per-item state: `docs/07_production_readiness.md`.

Next five, in order

1. Cross-compile in CI. ∼20 lines per kit. Until the firmware compiles for its target, “complete” is a claim about structure. Nothing else on this list matters more.

2. Validate on CWRU and MFPT with bearing-wise partitioning. Data is public, harness exists. Until then every detection number here is about a simulator.

3. Write the int8 export tool. The inference path is pinned by golden vectors; nothing converts a trained detector into the C arrays. That is the missing link between training and the device.

4. Wire ingest to Postgres. Schema complete, service holds work orders in memory. No transactional write means no audit trail.

5. Adopt arXiv 2607.08273’s regime-based split and residual calibration. Closest published work to this problem, MIT-licensed.